

Mini coding project 1

Part A: key skills in RStudio (8 marks)

Question 1 (3 marks total)

Write R code to achieve the following:

- a. 100 plus 5, all divided by 2

Solution.

You need to use brackets around `100 + 5` as otherwise with the order of operations R will do `100 + 2.5` (divide 5 by 2 first)

```
(100 + 5) / 2
```

```
[1] 52.5
```

- b. Rounding pi to 3 decimal places

Solution.

We can use the `round()` function, and we need to adjust the `digits` argument to 3 (if you forget the argument name, run `?round` in the console to bring up the help page)

```
round(pi, digits = 3)
```

```
[1] 3.142
```

- c. Testing whether letters and pi have the same atomic type (class)

Solution.

Recall that you find atomic type with the `class()` function. To test if they are the same, we can use the comparison operator `==` (remember that to test equality you need the double equal signs!!)

```
class(letters) == class(pi)
```

[1] FALSE

Question 2 (3 marks total)

I've tried to write the following code, but it gives me an error:

```
cancer properties <- c("status", "thickness")
```

```
Error in parse(text = input): <text>:1:8: unexpected symbol
1: cancer properties
   ^
```

- a. Explain in words why there is an error (1 mark)

Solution.

The error occurs because the variable name has a space in it. You should either use `snake_case` or `camelCase` for variable names containing multiple words.

- b. In the code chunk below, correct the mistake (2 marks)

Solution.

Either of the following are ideal:

```
cancer_properties <- c("status", "thickness")
cancerProperties <- c("status", "thickness")
```

Note that the code would still run if you simply removed the space and used `cancerproperties`, but this is harder to read and not recommended.

Question 3 (2 marks)

I want to find the mean (average) of the numbers 1, 2, 3, 4 and 5 using R. I tried using the following code:

```
mean(1, 2, 3, 4, 5)
```

```
[1] 1
```

We know the mean of 1,2,3,4,5 should be 3, but running this code in RStudio gives the wrong answer.

- a. Explain why (1 mark)

Solution.

The code `mean(1, 2, 3, 4, 5)` returns the value 1. This is because the `mean()` function expects a **vector**, as its first argument, `x`. So, what it's actually doing here is calculating the mean of the singular value 1, which is of course 1.

- b. In the code chunk below, correct the mistake (1 mark)

Solution.

The numbers 1-5 need to be in a vector like so:

```
mean(c(1, 2, 3, 4, 5))
```

```
[1] 3
```

It's also acceptable to use the shorthand colon notation we learned in workshop 1 like so:

```
mean(1:5)
```

```
[1] 3
```

Part B: analysing biomedical datasets (12 marks)

Question 5 (2 marks)

Inside this assessment's R project, there is a folder called 'data_for_part_b', that contains a file called 'melanoma_dataset.tsv'. Please write code to read this data into R, so that you can use it to answer the subsequent questions.

Solution.

There are three things you need to keep in mind when answering this question:

1. The path to the file. Seeing as we're in an R project, you can use the relative path "data_for_part_b/melanoma_dataset.tsv" or use the gear button in the files panel to get the full path from your computer. Mine is "~/Desktop/teaching/mini_coding_project_1/data_for"
2. Use one of the readr functions to read in the file. As the file is a .tsv, we should use read_tsv() or read_delim()
3. Assign the data to an appropriately named variable. Here I have called mine melanoma_data, but you can choose whatever you like

Putting it together, we get the code:

```
melanoma_data <- read_tsv("data_for_part_b/melanoma_dataset.tsv")
```

Rows: 205 Columns: 7

-- Column specification -----

Delimiter: "\t"

dbl (7): time, status, sex, age, year, thickness, ulcer

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

You may also use the RStudio GUI's import dataset button, **so long as you copy the code into the script.**

Question 6 (1 mark)

How many rows and columns are in the melanoma_data dataframe?

Solution.

There are 205 rows and 7 columns.

To get this you could use the `nrow()` and `ncol()` functions like so:

```
nrow(melanoma_data)
```

```
[1] 205
```

```
ncol(melanoma_data)
```

```
[1] 7
```

Other acceptable answers include:

```
# the dim() function gives the number of rows and number of columns  
dim(melanoma_data)
```

```
[1] 205 7
```

```
# the length() function gives the number of columns of a data frame  
length(melanoma_data)
```

```
[1] 7
```

```
# and the length of a column is equivalent to the number of rows  
length(melanoma_data$time)
```

```
[1] 205
```

Or anything else that gives the correct answer through R code, as long as it is clear (through text or comments) which one is the rows and which one is the columns.

Question 7 (2 marks total)

The `melanoma_data` dataframe contains information about patients with melanoma. One piece of information that has been recorded is the age of the patients.

- a. What is the range of ages of patients in this study?

Solution.

The range is between 4 and 95 years.

You first need to access the age column, using the `$` operator. Then, you could use the `range()` function:

```
range(melanoma_data$age)
```

```
[1] 4 95
```

Or you could use the `min()` and `max()` functions like so:

```
min(melanoma_data$age)
```

```
[1] 4
```

```
max(melanoma_data$age)
```

```
[1] 95
```

You could also use any of those functions in a more 'tidyverse' way, using the `pull()` function and the pipe `%>%` like so:

```
melanoma_data %>%  
  pull(age) %>%  
  range()
```

```
[1] 4 95
```

- b. What is the average age of patients in this study?

Solution.

The average age is 52.46341 (also accept any rounding of this number e.g. 52 or 52.46)

You need to use the `mean()` function, either with the `$` or `pull()` to access the age column:

```
# using $
mean(melanoma_data$age)
```

```
[1] 52.46341
```

```
# using pull()
melanoma_data %>%
  pull(age) %>%
  mean()
```

```
[1] 52.46341
```

Question 8 (2 marks total)

- a. By default, this data is sorted by the time column (the number of days patients survived after surgery to remove their melanoma). What is the fewest number of days survived by a patient?

Solution.

The fewest number of days survived is 10.

There are two main ways we can achieve this. Since we know the data is already sorted in the way we require, we could simply use indexing:

```
melanoma_data$time[1]
```

```
[1] 10
```

Or, we could use the `slice_min()` function from `dplyr`:

```
# slice_max() on 'thickness' column, with n=3 to get top 3
melanoma_data %>%
  slice_min(time, n = 1)
```

```
time status sex age year thickness ulcer
1 10      3  1  76 1972      6.76    1
```

Or, we could use the `head()` function:

```
melanoma_data %>%
  head(n = 1)
```

```
time status sex age year thickness ulcer
1 10      3  1  76 1972      6.76    1
```

Amongst many other solutions!

- b. The researchers who collected this data are interested in how thick a melanoma can possibly be. What are the top 3 thickest tumours in this data? (note: the units of measurement for the thickness column are mm)

Solution.

The three thickest tumours are 17.42, 14.66 and 13.85 millimeters.

There are two main ways we can achieve this. Firstly, we could use the `slice_max()` function from `dplyr`:

```
# slice_max() on 'thickness' column, with n=3 to get top 3
melanoma_data %>%
  slice_max(thickness, n = 3)
```

```
time status sex age year thickness ulcer
43 1312     1  0  65 1970      17.42    1
35 1041     1  1  68 1967      14.66    0
21  667     1  0  89 1968      13.85    1
```

Or, we can combine the `arrange()` function with `head()`:

```
melanoma_data %>%
  # note that we need to use desc() here to put them in descending order
  arrange(desc(thickness)) %>%
  head(n = 3)
```

```
time status sex age year thickness ulcer
43 1312     1  0  65 1970      17.42    1
35 1041     1  1  68 1967      14.66    0
21  667     1  0  89 1968      13.85    1
```


Question 9 (1 mark)

The 'ulcer' column indicates whether the tumour was ulcerated or not. It is encoded as 1 = presence of ulcer, 0 = absence of ulcer. How many of the tumours were ulcerated?

Solution.

There are 90 ulcerated tumours.

To determine this, we need to use the `filter()` function, with the comparison `ulcer == 1` (don't forget the double equal sign!)

```
# use filter to check if 'ulcer' is equal to 1
melanoma_data %>%
  filter(ulcer == 1)
```

	time	status	sex	age	year	thickness	ulcer
1	10	3	1	76	1972	6.76	1
5	185	1	1	52	1965	12.08	1
6	204	1	1	28	1971	4.84	1
7	210	1	1	77	1972	5.16	1
8	232	3	0	60	1974	3.22	1
9	232	1	1	49	1968	12.88	1
10	279	1	0	68	1971	7.41	1
11	295	1	0	53	1969	4.19	1
12	355	3	0	64	1972	0.16	1
13	386	1	0	68	1965	3.87	1
14	426	1	1	63	1970	4.84	1
15	469	1	0	14	1969	2.42	1
16	493	3	1	72	1971	12.56	1
17	529	1	1	46	1971	5.80	1
18	621	1	1	72	1972	7.06	1
19	629	1	1	95	1968	5.48	1
20	659	1	1	54	1972	7.73	1
21	667	1	0	89	1968	13.85	1
22	718	1	1	25	1967	2.34	1
23	752	1	1	37	1973	4.19	1
24	779	1	1	43	1967	4.04	1
25	793	1	1	68	1970	4.84	1
27	826	3	0	86	1965	8.54	1
28	833	1	0	56	1971	2.58	1
32	967	1	1	52	1970	4.83	1
33	977	1	1	58	1967	1.62	1

34	982	1	0	60	1970	6.44	1
36	1055	1	0	75	1967	2.58	1
37	1062	1	1	19	1966	3.87	1
38	1075	1	1	66	1971	3.54	1
39	1156	1	0	56	1970	1.34	1
40	1228	1	1	46	1973	2.24	1
41	1252	1	0	58	1971	3.87	1
42	1271	1	0	74	1971	3.54	1
43	1312	1	0	65	1970	17.42	1
47	1506	1	1	56	1970	4.51	1
48	1508	2	1	63	1973	8.38	1
51	1516	1	1	80	1968	2.58	1
52	1525	3	0	76	1970	1.29	1
60	1621	1	0	46	1972	5.16	1
69	1667	1	0	44	1971	5.80	1
75	1726	1	0	58	1970	0.97	1
76	1745	2	0	47	1973	2.58	1
78	1779	2	1	55	1973	3.54	1
80	1787	2	0	41	1973	1.78	1
83	1812	2	1	44	1973	3.22	1
86	1839	2	1	53	1972	1.62	1
87	1854	2	0	65	1972	1.62	1
89	1860	3	1	71	1969	4.84	1
94	1920	2	1	60	1972	1.62	1
95	1927	2	1	40	1972	2.58	1
98	1955	2	0	46	1972	7.73	1
102	1970	2	1	84	1972	4.09	1
105	2011	2	0	75	1972	3.22	1
107	2028	2	1	52	1972	3.87	1
108	2038	2	0	58	1972	0.32	1
110	2059	2	1	68	1972	3.22	1
113	2075	2	1	55	1972	2.58	1
118	2108	1	0	58	1969	1.76	1
119	2112	2	0	54	1972	1.94	1
125	2227	2	1	77	1971	5.48	1
126	2256	1	0	43	1971	2.26	1
127	2264	2	0	65	1971	4.83	1
130	2387	2	0	50	1971	5.16	1
132	2403	2	0	67	1971	2.90	1
134	2426	2	0	74	1971	1.94	1
137	2467	1	0	42	1965	2.26	1
139	2493	2	1	72	1971	4.82	1
140	2521	2	0	45	1971	1.29	1

141	2542	2	1	67	1971	7.89	1
142	2559	2	0	48	1970	0.81	1
143	2565	1	1	34	1970	3.54	1
146	2666	2	0	42	1970	3.22	1
147	2676	2	0	24	1970	1.45	1
154	3042	1	0	54	1967	3.54	1
165	3228	2	0	58	1969	3.22	1
168	3297	2	0	61	1968	2.58	1
170	3330	2	1	61	1968	0.81	1
171	3338	1	0	60	1967	3.22	1
173	3384	2	0	68	1968	3.22	1
175	3388	2	1	60	1968	4.84	1
180	3459	2	1	56	1968	1.29	1
186	3776	2	1	12	1967	7.09	1
187	3776	2	0	21	1967	1.29	1
190	3872	2	0	35	1967	12.24	1
191	3909	2	1	42	1967	8.06	1
196	4124	2	0	30	1966	1.94	1
199	4390	2	0	26	1965	1.94	1
200	4479	2	0	19	1965	1.13	1
201	4492	2	1	29	1965	7.06	1

We could then read the number of rows, or optionally pipe into `nrow()` so R does this for us:

```
# use filter to check if 'ulcer' is equal to 1
melanoma_data %>%
  filter(ulcer == 1) %>%
  # then use nrow() to count how many rows
  nrow()
```

[1] 90

Question 10 (4 marks total)

The researchers are interested specifically in the melanoma of patients born after 1920. However, they have only recorded the age of the patients at diagnosis (age column) and the year in which they were diagnosed (year column).

- Looking at the `melanoma_data` dataframe, **explain in words** how you could find birth year from this data (1 mark)

Solution.

We can find the birth year by subtracting the age of the patients from the year the data was collected, i.e. `year - age`

- b. Write R code to create a column called `birth_year` in `melanoma_data`. Ensure that the result is saved as a variable (2 marks)

Solution.

You can either use the `$` and `<-` syntax we covered in workshop 1:

```
melanoma_data$birth_year <- melanoma_data$year - melanoma_data$age
```

Or the `mutate()` syntax we covered in workshop 2:

```
melanoma_data <- melanoma_data %>%  
  mutate(birth_year = year - age)
```

- c. Using your column from part b, how many patients were born after 1920? (1 mark)

Solution.

The easiest way to do this would be to use the `filter()` function together with the `nrow()` function:

```
melanoma_data %>%  
  filter(birth_year > 1920) %>%  
  nrow()
```

`[1] 80`